SPL Compiler Front-End: User Manual

COS341 Project Phase 1 · Group 21

1. Group members

Full name	Student number
Antony Van Straten	24590739
Keegan Lewis	24594522
Aidan Dawson	24593542
Sofia Finlayson	24593240

2. What the program does

The program is the front-end of our SPL compiler: lexical analysis combined with syntax analysis. It reads one SPL program from a plain text file and reports whether the program is valid.

- If the program is **accepted**, it prints a confirmation and the parse tree, and writes the syntax tree to a file named `tree.xml`.
- If the program is **rejected**, it prints the first error it found, with the line and column number and a marker under the offending text. No `tree.xml` is written.

The executable is supplied as a runnable Java archive, `group-21.jar`. It has no graphical interface and is run from a terminal.

3. Requirements

Java 21 or newer must be installed. No other software is needed; all libraries the program uses are packaged inside `group-21.jar`.

To check the installed Java version, open a terminal (see section 5) and run:

```
java -version
```

The first line of the output shows the version, for example `openjdk version "21.0.4"`. The number must be 21 or higher. If Java is missing or older than 21, install Java 21 (for example Eclipse Temurin 21 from adoptium.net) and open a new terminal before continuing.

4. Contents of the ZIP file

File	Description
`group-21.jar`	The executable front-end (runnable Java archive)
`group-21.pdf`	This user manual

5. Running the program

1. Extract `group-21.zip` into a folder.
2. Open a terminal in that folder. **Windows:** open the folder in File Explorer, click the address bar, type `cmd` and press Enter. **macOS:** open Terminal (Applications → Utilities → Terminal), type `cd` followed by a space, drag the folder into the Terminal window and press Enter.
3. Run the program, giving the SPL test file as the only argument:

```
java -jar group-21.jar test1.txt
```

4. Repeat step 3 for each test file.

Notes on the input file

- The file path can be relative to the folder the terminal is in, or a full path. If the path contains spaces, put it in double quotes, for example `java -jar group-21.jar "C:\SPL Tests\test1.txt"` on Windows or `java -jar group-21.jar "/Users/tutor/SPL Tests/test1.txt"` on macOS.
- The file must be plain text containing one SPL program. Any file name or extension works.
- The end of the file marks the end of the program. The pseudo-symbol `$` is not written in the file.
- Do not double-click `group-21.jar`. It is a console program, so double-clicking opens no window and it will appear that nothing happened.

6. Reading the output

Accepted program

```
Parsed OK: valid1_minimal.txt
Parse tree: (spl_prog (p v_decl : f_decl : (algo (instr print (outp "hello")) ; algo))
<EOF>)
Outputting the XML now
Check /Users/tutor/SPL Tests/tree.xml
```

`Parsed OK` means the program passed both lexical and syntax analysis. The `Parse tree` line shows the same tree in bracketed text form, and the last line gives the full path of the `tree.xml` file that was written. `<EOF>` marks the end of the input file.

Rejected program: lexical error

A lexical error means the file contains text that cannot form any SPL token, such as an illegal symbol, a misspelt keyword, a variable name without #, or an unterminated string:

```
Lexing error at line 3, column 7: token recognition error at: '@'
  print @oops ;
        ^
```

Rejected program: syntax error

A syntax error means every token is valid, but they are not in an order the SPL grammar allows. The `hint` line lists what the parser expected at that point:

```
Parsing error at line 4, column 6: missing '{' at 'print'
  hint: expected '{'
  then print "zero"
```

```
       ^
```

In both cases, line and column numbers start at 1, and the ^ marker points to where the error was found.

Input file not found

```
Input file not found: /Users/tutor/SPL Tests/tset1.txt
Usage: java -jar group-21.jar <path-to-SPL-file>
```

The message shows the full path the program tried, which makes a misspelt name or wrong folder easy to spot.

7. The syntax tree file (tree.xml)

- `tree.xml` is written to the folder the terminal is in when the command is run, not necessarily the folder containing the jar.
- It is only written when a program is accepted, and each accepted run overwrites the previous file. A rejected program leaves any older `tree.xml` untouched, so delete it before each test to avoid confusing an old tree with a new result.

The file has one `root` node (`spl_prog`), `inner` nodes for non-terminals, and `leaf` nodes for terminals (the tokens of the program). Every node has a unique `id`; every node except the root has a `parent`; inner nodes list their children by id. A non-terminal that derives the empty string appears as an inner node with an empty `<children/>` element. An excerpt:

```
<syntaxTree>
  <root>
    <id>0</id>
    <contents>spl_prog</contents>
    <children>1</children>
  </root>
  <inner>
    <id>1</id>
    <contents>p</contents>
    <children>2 21 22 165 166</children>
    <parent>0</parent>
  </inner>
  <leaf>
    <id>3</id>
    <contents>#x</contents>
    <parent>2</parent>
  </leaf>
  ...
</syntaxTree>
```

8. Troubleshooting

Message or symptom	Cause	Fix
`'java' is not recognized` (Windows) or `command not found: java` (macOS)	Java is not installed, or not on the PATH	Install Java 21 or newer, then open a new terminal
`UnsupportedClassVersionError`	The installed Java is older than 21	Install Java 21 or newer
`Unable to access jarfile group-21.jar`	The terminal is not in the folder containing the jar	Change to that folder with `cd`, or give the full path to the jar
`Input file not found: …`	The test file path is wrong	Check the spelling and folder; quote paths that contain spaces
Double-clicking the jar does nothing	It is a console program	Run it from a terminal as described in section 5